



Học viện Công nghệ Bưu chính Viễn thông
Khoa Công nghệ thông tin 1

Phân tích và khai phá dữ liệu văn bản

Chương 5: Độ tương tự trong văn bản và phân cụm

TS. Nguyễn Thị Mai Trang
trangntm@ptit.edu.vn

Độ Tương Tự Văn Bản

- **Độ Tương Tự Văn Bản** định lượng mức độ gần hoặc liên quan giữa hai đoạn văn bản (từ, câu, tài liệu).
- **Mục tiêu:** Chuyển so sánh ngôn ngữ định tính thành điểm số định lượng (thường từ 0 đến 1, hoặc một chỉ số khoảng cách).
 - Độ Tương Tự -> Điểm cao,
 - Khoảng Cách -> Điểm thấp

Ứng dụng của Độ Tương Tự

- **Truy xuất Thông tin (IR):** Xếp hạng kết quả tìm kiếm
- **Trả lời Câu hỏi (QA):** Khớp câu hỏi với câu trả lời đã lưu trữ hoặc ngữ cảnh liên quan.
- **Phát hiện Đạo văn:** So sánh các tài liệu đã nộp.
- **Hệ thống Gợi ý:** Tìm các bài viết/sản phẩm tương tự với sở thích của người dùng.
- **Phân cụm & Phân loại:** Nhóm và dán nhãn các tài liệu dựa trên nội dung.



Tổng quan về Độ Tương Tự Từ Vựng

- So sánh các chuỗi ký tự riêng lẻ (từ hoặc chuỗi ngắn) dựa trên sự khác biệt về ký tự hoặc vị trí.
- **Ngữ cảnh:** Hữu ích cho kiểm tra chính tả, khớp mờ (fuzzy matching) và so sánh ngữ âm.
- **Các Chỉ số được đề cập:** Hamming, Manhattan, Euclidean, Levenshtein, Cosine.



Khoảng cách Hamming

- **Định nghĩa:** Số lượng vị trí mà các ký hiệu tương ứng khác nhau.
- **Ràng buộc:** Chỉ được định nghĩa cho các chuỗi có độ dài bằng nhau.
- **Công thức:**

Giả sử S_1 và S_2 là hai chuỗi có độ dài N . Khoảng cách Hamming giữa S_1 và S_2 được định nghĩa là:

$$D_{Hamming}(S_1, S_2) = \sum_{i=1}^N [S_1[i] \neq S_2[i]]$$

trong đó biểu thức $[S_1[i] \neq S_2[i]] = 1$ nếu các ký tự của S_1 và S_2 khác nhau, và ngược lại bằng 0.

Ví dụ Khoảng cách Hamming

Các chuỗi: $X = \text{karolin}$, $Y = \text{kathrin}$

Yêu cầu: *tính khoảng cách Hamming của 2 chuỗi*

$X = \text{karolin}$

$Y = \text{kathrin}$

$$D_{\text{Hamming}}(X, Y) = 1 + 1 + 1 = 3$$

Khoảng cách Manhattan

- **Định nghĩa:** Khoảng cách Manhattan là tổng các trị tuyệt đối của hiệu số giữa các tọa độ Descartes của chúng (còn gọi là *Chuẩn L1/ khoảng cách Taxicab*).
- **Ứng dụng cho văn bản:** Được sử dụng sau khi chuyển đổi văn bản thành Mô hình không gian vector, thường sử dụng túi từ (Bag-of-Words).
- **Công thức:**

Cho 2 vectors $A=(a_1,a_2,...,a_n)$ và $B=(b_1,b_2,...,b_n)$:

$$D_{Manhattan}(A, B) = \sum_{i=1}^N |a_i - b_i|$$



Ví dụ Khoảng cách Manhattan

Giả sử ta có 4 từ trong từ điển:

`["dog", "cat", "car", "bus"]`

Biểu diễn one-hot cho "dog" và "cat". Tính khoảng cách Manhattan giữa 2 từ này.

dog: [1,0,0,0]

Cat: [0,1,0,0]

$D_{\text{Manhattan}}(\text{dog}, \text{cat})$

$= |1-0| + |0-1| + |0-0| + |0-0|$

$= 1+1$

$= 2$

Ví dụ Khoảng cách Manhattan

Giả sử ta có hai câu ngắn:

A = "I like AI"

B = "I love AI"

Tính khoảng cách Manhattan cho A và B

Vector A = [1, 1, 0, 1]

Vector B = [1, 0, 1, 1]

$D(A, B) = 2$

Khoảng cách Euclidean

- **Định nghĩa:** Căn bậc hai của tổng bình phương hiệu số các tọa độ. (Còn gọi là Chuẩn L2).
- **Ứng dụng cho văn bản:** Cũng được áp dụng sau khi chuyển đổi văn bản thành Vector đếm từ hoặc vector TF-IDF. Rất nhạy cảm với sự khác biệt về độ lớn (độ dài tài liệu).
- **Công thức:** Cho 2 vector $A=(a_1,a_2,...,a_n)$ và $B=(b_1,b_2,...,b_n)$:

$$D_{Euclidean}(A, B) = \sqrt{\sum_{i=1}^N (a_i - b_i)^2}$$

Ví dụ Khoảng cách Euclidean

Sử dụng cùng các vector số lần xuất hiện ký tự:

A ("apple"): (1, 2, 1, 1, 0)

B ("apply"): (1, 2, 1, 0, 1)

Tính toán:

$$\mathbf{d}_E(A, B) = \sqrt{2}$$

$$\mathbf{d}_{\text{Manhattan}}(\mathbf{A}, \mathbf{B}) = 2$$

Lưu ý: Khoảng cách thường thấp hơn khoảng cách Manhattan vì các hiệu số được bình phương trước khi tính tổng.

Khoảng cách Levenshtein

- **Khoảng cách Levenshtein** là số lượng tối thiểu các **thao tác chỉnh sửa** một ký tự (chèn, xóa hoặc thay thế) cần thiết để biến từ này thành từ kia.
- Chỉ số phổ biến nhất để đo độ tương tự chuỗi khi cho phép biến đổi độ dài và vị trí.
- Tiếp cận Lập trình Động (Dynamic Programming) sử dụng ma trận chi phí.

Công thức tính khoảng cách Levenshtein

Khoảng cách $d(a, b)$ giữa các chuỗi a và b được tính toán đệ quy.

Hàm chi phí $c(a_i, b_j)$ là 0 nếu $a_i = b_j$, là 1 nếu ngược lại.

Bước Đệ quy:

$$d_{i,j} = \min \begin{cases} d_{i-1,j} + 1 & (\text{Xóa}) \\ d_{i,j-1} + 1 & (\text{Chèn}) \\ d_{i-1,j-1} + c(a_i, b_j) & (\text{Thay thế}) \end{cases}$$



Ví dụ “KITTEN” so với “SITTING”

Tính toán từng bước: Đi sâu vào quá trình điền ma trận.

K → S: Thay thế (1) / KITTEN → SITTEN

I → I: Không chi phí (0)

T → T: Không chi phí (0)

T → T: Không chi phí (0)

E → I: Thay thế (1) / SITTEN → SIT TIN

N → N: Không chi phí (0)

Chèn G (1) / SIT TIN → SITTING

Kết quả: Thay 2 ký tự, thêm 1 ký tự

$d(\text{KITTEN}, \text{SITTING}) = 3$

```
import Levenshtein
Levenshtein.distance("KITTEN", "SITTING")
```





Độ Tương Tự và Khoảng cách Cosine

- **Ngữ cảnh:** Đo cosin của góc giữa hai vector khác 0.
- Đo **hướng**, không phải độ lớn. Hai tài liệu có thể rất giống nhau (góc nhỏ) ngay cả khi một tài liệu dài hơn nhiều (độ lớn lớn hơn).
- **Phạm vi Tương Tự:** $[-1, 1]$. Trong văn bản, thường là $[0, 1]$ (vì tần suất thuật ngữ là không âm).
- **Khoảng Cách:**
 - Cosine Distance = $1 - \text{Cosine Similarity}$





Độ Tương Tự và Khoảng cách Cosine

$$S_{Cosine}(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Ví dụ:

Cho 2 vector A,B

$A = (3, 2, 0, 5, 0)$

$B = (1, 0, 0, 1, 0)$

Tính độ tương tự cosine giữa 2 vector A,B





Bài tập 1

Cho 3 chuỗi X, Y, Z

X: ANALOGY

Y: ANTHOLOGY

Z: ANALOG

Hãy:

- a) Tính khoảng cách **Hamming** giữa X và Y.
- b) Tính khoảng cách **Levenshtein** giữa X và Z



Bài tập 2

Viết chương trình Python tính các loại khoảng cách giữa các chuỗi văn bản.

Dữ liệu:

```
text1 = "I love deep learning"
```

```
text2 = "I enjoy machine learning"
```

```
text3 = "Deep learning is amazing"
```

Yêu cầu:

- Biểu diễn các câu bằng TF-IDF hoặc CountVectorizer
 - Tính các khoảng cách giữa từng cặp câu:
 - Manhattan Distance
 - Euclidean Distance
 - Cosine Similarity
 - Tính Levenshtein Distance cho các cặp từ:
("KITTEN", "SITTING"), ("BOOK", "BACK"), ("NLP", "DEEP")
 - Viết hàm `compare_texts(text1, text2)` trả về kết quả 4 loại khoảng cách.
 - Hiển thị kết quả bằng bảng Pandas DataFrame.
-

Độ Tương Tự Văn Bản so với Từ Vựng

- **Từ vựng:** Cấp độ ký tự, khớp chuỗi.
- **Văn bản:** Dựa trên tần suất và tầm quan trọng của các **thuật ngữ** (từ) trong tài liệu. Yêu cầu vector hóa.
- **Vector Space Model (VSM):** Các tài liệu được biểu diễn dưới dạng các vector tần suất thuật ngữ có trọng số (ví dụ BoW, TF-IDF,...)

Cosine Similarity (Deep Dive - VSM)

- Thước đo được sử dụng rộng rãi nhất trong Truy xuất Thông tin.
- Khi được sử dụng với các vectơ TF-IDF, nó đo lường sự liên kết ngữ nghĩa (mức độ liên quan của các chủ đề), chứ không chỉ là sự chồng chéo từ.
- Diễn giải Hình học:
 - Điểm tương đồng bằng 1 có nghĩa là các vectơ giống hệt nhau (góc bằng 0).
 - Điểm bằng 0 có nghĩa là các vectơ trực giao (góc bằng 90 độ- không có thuật ngữ chung).
 - Điểm -1 có nghĩa là các vectơ ngược nhau (góc bằng 180 độ - hiếm gặp trong VSM).

Ví dụ TF-IDF + Cosine Similarity

Tính TF-IDF và Cosine Similarity giữa hai tài liệu sau:

D₁: "The quick brown fox."

D₂: "The fast brown dog."

→ Vector TF-IDF:

• $D_1 = (0.693, 0, 0.693, 0, 0)$

• $D_2 = (0, 0, 0, 0.693, 0.693)$

Tính Cosine Similarity

$$\text{cosine}(D_1, D_2) = \frac{D_1 \cdot D_2}{\|D_1\| \|D_2\|}$$

$$0.693 \times 0 + 0 \times 0 + 0.693 \times 0 + 0 \times 0.693 + 0 \times 0.693 = 0$$

Cosine Similarity = 0

Hai câu không có từ khóa trùng (ngoài "brown" – nhưng IDF=0 → không phân biệt), vì vậy chúng **không tương tự nhau** (similarity = 0).

Khoảng cách Hellinger-Bhattacharya (H-B)

- Không giống như khoảng cách Euclidean hoặc Cosine, HBD chuyên dùng để so sánh hai phân phối xác suất.
- Yêu cầu: Các vector đầu vào P và Q phải biểu diễn các phân phối xác suất (tức là các phần tử không âm và có tổng bằng 1).
- Trường hợp sử dụng trong NLP: So sánh các Phân phối Chủ đề (ví dụ: từ LDA), so sánh các phân phối nhúng từ (Word Embedding), hoặc các vector tài liệu được chuẩn hóa để tổng bằng 1.

Khoảng cách Hellinger-Bhattacharya (H-B)

- Khoảng cách HB có thể tính toán được cho cả phân phối xác suất liên tục và rời rạc.
- Có thể định nghĩa khoảng cách HB về mặt toán học như sau:

$$hbd(u, v) = \frac{1}{\sqrt{2}} \|\sqrt{u} - \sqrt{v}\|_2$$

Khoảng cách HB giữa 2 tài liệu u , v bằng chuẩn Euclid hoặc chuẩn L2 của hiệu số giữa căn bậc hai của các vector chia cho căn bậc hai của 2.

- Xem xét các vector tài liệu u và v là rời rạc với n số đặc trưng, chúng ta có thể mở rộng thêm công thức trên thành:

$$hbd(u, v) = \frac{1}{\sqrt{2}} \sqrt{\sum_{i=1}^n (\sqrt{u_i} - \sqrt{v_i})^2}$$



Khoảng cách Hellinger-Bhattacharya (H-B)

- Ví dụ:

Hãy tính khoảng cách HB cho 2 tài liệu:

D₁: "The quick brown fox."

D₂: "The fast brown dog."



Khoảng cách Hellinger-Bhattacharya (H-B)

```
def compute_hellinger_bhattacharya_distance(doc_features, corpus_features,
top_n=3):
    # get document vectors
    doc_features = doc_features.toarray()[0]
    corpus_features = corpus_features.toarray()
    # compute hb distances
    distance = np.hstack(
    np.sqrt(0.5 *
    np.sum(
    np.square(np.sqrt(doc_features) -
    np.sqrt(corpus_features)),
    axis=1)))
    # get docs with lowest distance scores
    top_docs = distance.argsort()[:top_n]
    top_docs_with_score = [(index, round(distance[index], 3))
    for index in top_docs]
    return top_docs_with_score
```

Okapi BM25

- **BM25** được sử dụng chủ yếu trong các công cụ tìm kiếm để ước tính mức độ liên quan của tài liệu đối với một truy vấn tìm kiếm nhất định. BM là viết tắt của Best Match.
- **BM25** không hoàn toàn là một thước đo độ tương tự, mà là một điểm số xếp hạng có hiệu quả cao.
- **Cải tiến so với TF-IDF: BM25** tính đến **độ dài tài liệu** (để ngăn tài liệu dài bị ưu tiên giả tạo) và sử dụng tỷ lệ tần suất thuật ngữ phi tuyến tính.

Okapi BM25

Điểm số cho tài liệu D với truy vấn Q là:

$$\text{Score}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{f(t, D) \cdot (k_1 + 1)}{f(t, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})}$$

Trong đó:

- $f(t, D)$: Tần suất thuật ngữ t trong tài liệu D
- $|D|$: Độ dài của tài liệu D (tính theo số từ)
- avgdl : Độ dài trung bình của các tài liệu trong tập hợp
- k_1 (giá trị điển hình: 1.2 - 2.0): Tham số điều khiển độ bão hòa cho $f(t, D)$
- b (giá trị điển hình: 0.75): Tham số điều khiển chuẩn hóa độ dài tài liệu

Okapi BM25

Cho dữ kiện như sau:

- Tập tài liệu có 3 văn bản, $avgdl = 100$ từ.
- Truy vấn Q: "new phone"
- Tham số: $k_1 = 1.2$, $b = 0.75$
- Tài liệu D_1 : $|D_1| = 100$ từ
- Tần suất thuật ngữ: $f(\text{"new"}, D_1) = 3$, $f(\text{"phone"}, D_1) = 2$
- $IDF(\text{"new"}) \approx 0.405$, $IDF(\text{"phone"}) \approx 1.098$

Tính điểm BM25 cho D_1 với Q

Bài tập

Viết chương trình trên Python thực hiện đồng thời ba nhiệm vụ sau:

- a) tính độ tương tự cosine giữa hai câu văn bản bất kỳ.
- b) tính khoảng cách Hellinger-Bhattacharya giữa các văn bản.
- c) tính điểm BM25 cho một tài liệu với một truy vấn, sử dụng các tham số và dữ liệu đầu vào cụ thể.

Phân cụm văn bản

K-means trong phân cụm văn bản

- Gom nhóm các văn bản có nội dung tương tự vào cùng một cụm sử dụng thuật toán K-means
- **Ứng dụng:** Phân nhóm tin tức, phát hiện chủ đề, tóm tắt dữ liệu lớn.
- **Ý tưởng chính:**
 - Chuyển văn bản $\rightarrow$ vector (TF-IDF, Word2Vec,...).
 - Áp dụng thuật toán K-Means để chia các vector thành K cụm.

Ví dụ: Tập tin tức chia thành $\rightarrow$ 3 cụm: Chính trị, Thể thao, Giải trí.

K-means trong phân cụm văn bản

Các bước thuật toán:

- Chọn ngẫu nhiên **K tâm cụm (centroids)**.
- **Gán mỗi điểm dữ liệu** vào cụm có tâm gần nhất (dựa trên khoảng cách Euclidean hoặc Cosine).
- **Cập nhật tâm cụm:** lấy trung bình các điểm trong cụm.
- Lặp lại cho đến khi tâm cụm ổn định.

Hàm Mục Tiêu trong K-Means

- **Mục tiêu của K-Means:**

Tìm cách chia tập dữ liệu thành K cụm sao cho các điểm trong cùng cụm càng **gần nhau** càng tốt.

- **Hàm mục tiêu cần tối thiểu hóa:**

$$J = \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

Trong đó:

- K : số cụm.
 - C_i : tập các điểm trong cụm thứ i .
 - μ_i : vector tâm cụm (centroid).
 - $\|x - \mu_i\|^2$: khoảng cách bình phương giữa điểm x và tâm cụm.
- J càng **nhỏ**, các cụm càng chặt chẽ.
 - K-Means lặp lại bước **gán cụm** và **cập nhật tâm** để giảm dần J

Đánh giá K-Means trong Phân cụm Văn bản

Chỉ số đánh giá:

- **Silhouette Score:**

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Trong đó:

- $a(i)$: khoảng cách trung bình từ i đến các điểm khác trong **cùng cụm**.
- $b(i)$: khoảng cách trung bình nhỏ nhất từ i đến các điểm của **cụm gần nhất khác**.

- Đo độ “gần” của một điểm với cụm của nó so với cụm khác)
- **Inertia (Within-Cluster Sum of Squares)**: tổng khoảng cách nội cụm.
- **Elbow Method**: chọn K tối ưu bằng cách tìm điểm gấp khúc.
 - Elbow tại $K=3 \rightarrow$ dữ liệu có thể được chia thành 3 nhóm chủ đề.

Đánh giá K-Means trong Phân cụm Văn bản

Chỉ số đánh giá:

- **Silhouette Score:**

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

- Đo độ “gần” của một điểm với cụm của nó so với cụm khác)
- **Inertia (Within-Cluster Sum of Squares):** tổng khoảng cách nội cụm.
- **Elbow Method:** chọn K tối ưu bằng cách tìm điểm gấp khúc.
 - Elbow tại $K=3 \rightarrow$ dữ liệu có thể được chia thành 3 nhóm chủ đề.

Lan truyền độ tương tự (Affinity propagation)

- Là phương pháp phân cụm **dựa trên trao đổi thông tin giữa các điểm dữ liệu**.
- Không cần xác định số cụm K trước (khác K-Means).
- Mỗi điểm có thể trở thành **trung tâm cụm (exemplar)** nếu được nhiều điểm khác “ủng hộ”.
- **Ứng dụng:**
Phân cụm văn bản, ảnh, tín hiệu, hoặc dữ liệu không biết trước số nhóm.

Ý tưởng chính của thuật toán

Cơ chế lan truyền:

Hai loại thông tin được truyền giữa các điểm:

- **Responsibility $r(i, k)$** : mức độ điểm k thích hợp làm trung tâm cho điểm i .
- **Availability $a(i, k)$** : mức độ điểm i nên chọn k làm trung tâm.

Quá trình:

- Cập nhật $r(i, k)$ và $a(i, k)$ lặp đi lặp lại cho đến khi hội tụ.
 - Điểm có tổng $r(k, k) + a(k, k)$ lớn nhất được chọn làm **trung tâm cụm**.
-

Cập nhật công thức

Cập nhật Responsibility:

$$r(i, k) = s(i, k) - \max_{k' \neq k} [a(i, k') + s(i, k')]$$

Cập nhật Availability:

$$a(i, k) = \begin{cases} \min(0, r(k, k) + \sum_{i' \notin \{i, k\}} \max(0, r(i', k))) & i \neq k \\ \sum_{i' \neq k} \max(0, r(i', k)) & i = k \end{cases}$$

Trong đó $s(i, k)$ là **độ tương tự**

Ưu, nhược điểm

Ưu điểm:

- Không cần xác định số cụm trước.
- Có thể phát hiện cụm có kích thước khác nhau.
- Thường hội tụ nhanh.

Nhược điểm:

- Cần chọn tham số “preference” (ảnh hưởng số cụm).
- Nhạy cảm với dữ liệu nhiễu.

Bài tập

Viết chương trình trên python sử dụng

a) thuật toán K-means

b) lan truyền độ tương tự

để phân cụm văn bản trên dữ liệu IMDb Movie data và trực quan hoá kết quả phân cụm.